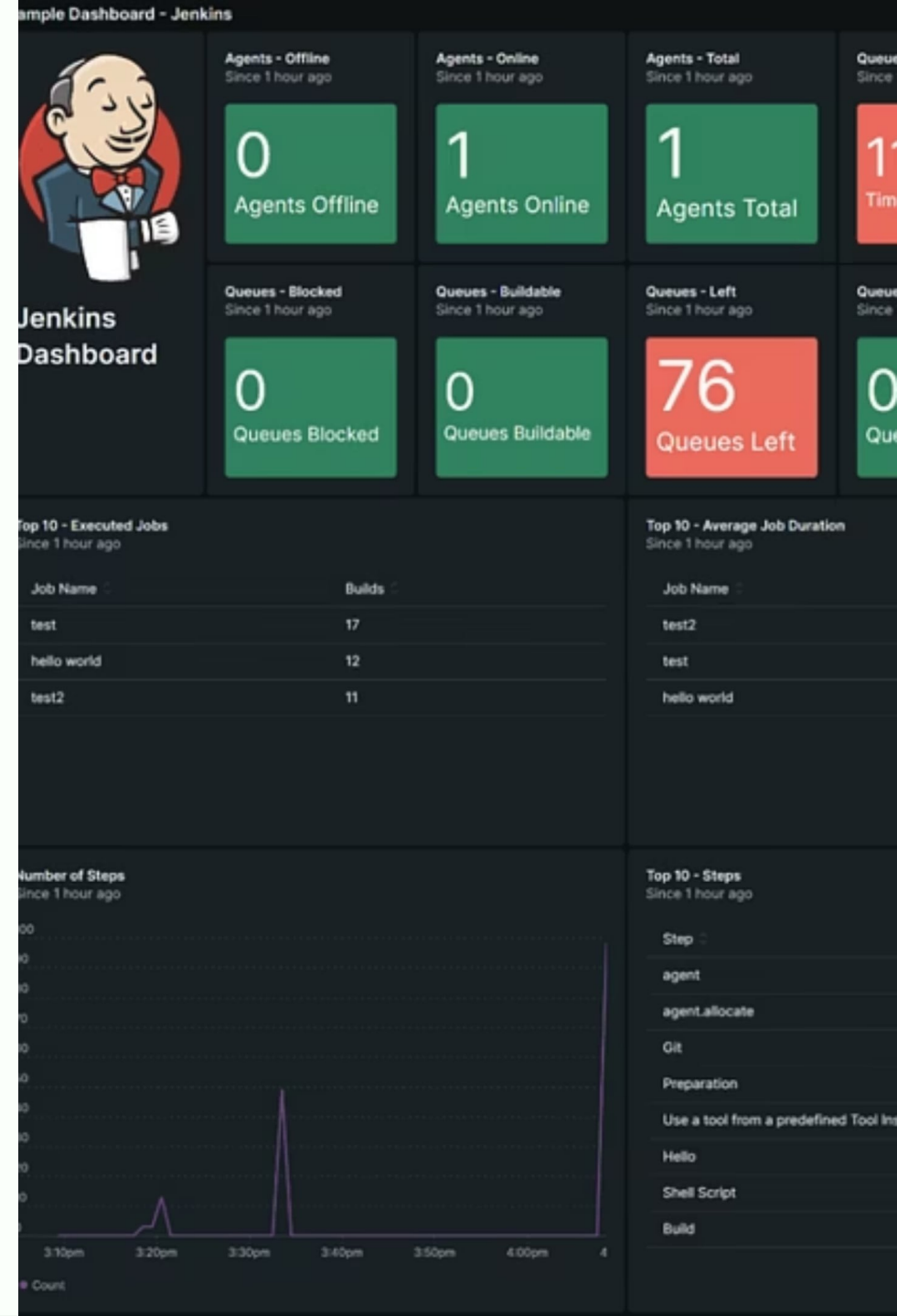


Jobs and Pipelines

Mastering Jenkins automation through jobs, pipelines, and best practices



Types of Jenkins Jobs

Jenkins offers three primary job types, each designed for different automation needs and project structures.



Freestyle Projects

Flexible, GUI-based jobs ideal for simple tasks and quick setup without coding requirements.



Maven Projects

Specialized jobs for Java builds with built-in Maven integration and dependency management.



Pipeline Projects

Code-based workflows offering advanced automation, version control, and complex orchestration capabilities.

Top Repositories

- 1 repo-1
- 3 repo-2
- repo-3
- 7 Other

Top Authors

- 138 someone
- 102 someone-else
- 85 another-one
- 422 Other

Branch	Build	Context	Status	
PR-2077	22	helm-chart-validation	Succeeded	C
PR-2077	21	build-sanitize-threads	Running	C
PR-2077	20	build-sanitize-memory	Running	C
PR-2077	19	build-sanitize-basic	Running	C
PR-2077	18	build-debug	Running	C
PR-2077	17	build-release	Running	C
PR-197	235	preview-env	Failed	C
PR-197	234	lint	Failed	C

Pipeline Syntax Approaches

Declarative Pipelines

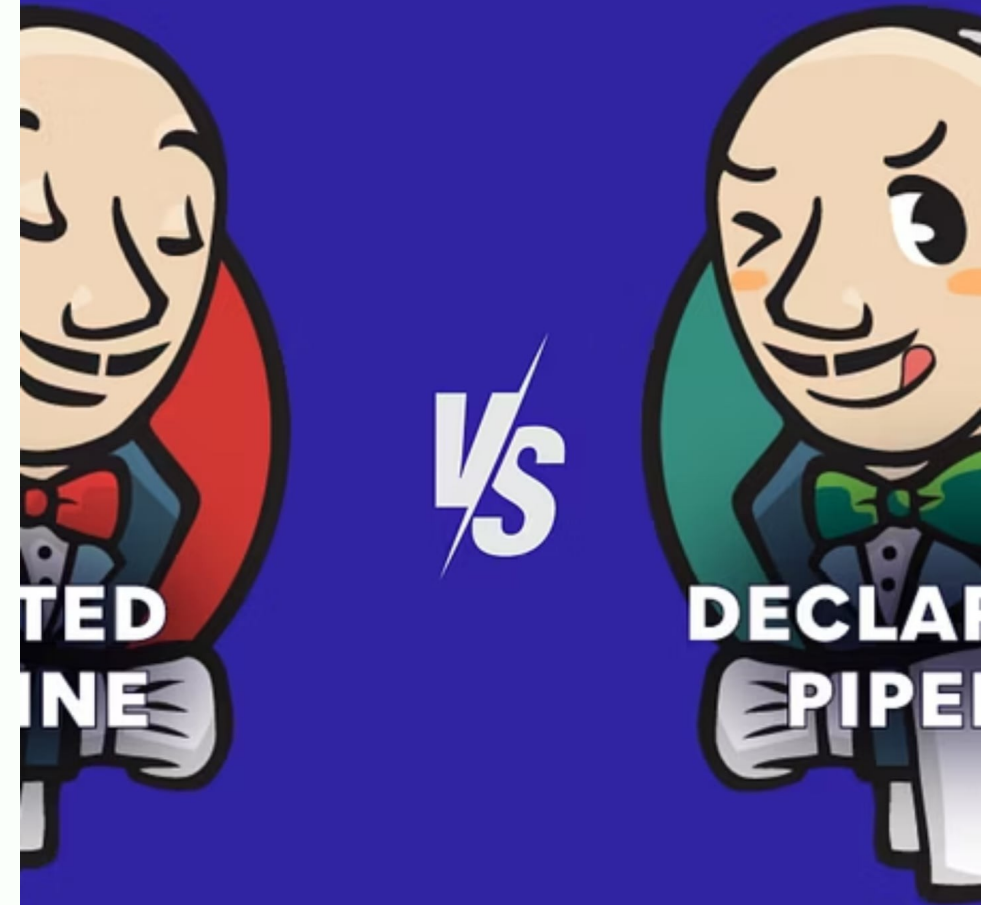
Structured, opinionated syntax with predefined sections. Easier to read and write, with built-in validation and error handling.

- Simplified syntax structure
- Better error messages
- Recommended for most use cases

Scripted Pipelines

Groovy-based, imperative syntax offering maximum flexibility. Ideal for complex workflows requiring advanced programming logic.

- Full Groovy programming power
- Greater customization options
- Steeper learning curve



Building with Jenkinsfile

Jenkinsfiles enable pipeline-as-code, storing your CI/CD configuration alongside your source code for version control and collaboration.

01

Create Jenkinsfile

Define your pipeline in a text file named "Jenkinsfile" at your repository root.

03

Configure Jenkins

Point your Jenkins job to the repository containing the Jenkinsfile.

02

Commit to Repository

Store the Jenkinsfile in version control alongside your application code.

04

Execute Pipeline

Jenkins automatically reads and executes the pipeline definition from your Jenkinsfile.

Pipeline Building Blocks

Stages



Logical divisions of your pipeline representing major phases like Build, Test, and Deploy. Stages provide visual organization in the Jenkins UI.

Steps

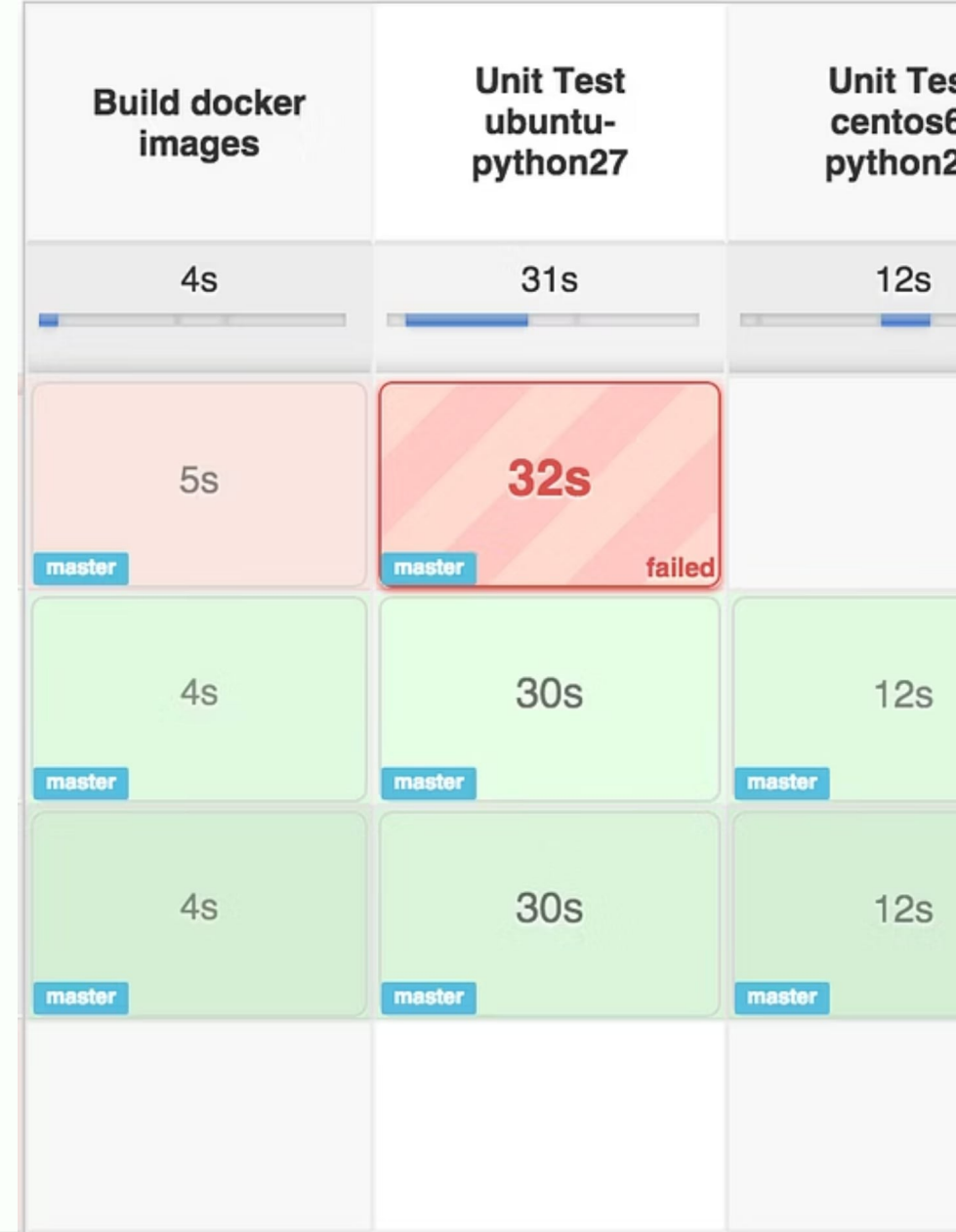


Individual tasks executed within stages, such as shell commands, script execution, or plugin invocations. Steps are the actual work units.

Environment Variables



Configuration values accessible throughout your pipeline, including credentials, paths, and custom settings for different environments.



Advanced Pipeline Features



Input Steps

Pause pipeline execution to request manual approval or user input before proceeding to critical stages like production deployment.



Parallel Execution

Run multiple stages or steps simultaneously to reduce total pipeline execution time and maximize resource utilization.



Post-Build Actions

Execute cleanup tasks, send notifications, archive artifacts, or trigger other jobs after pipeline completion, regardless of success or failure.

Input Steps in Action

Input steps provide human checkpoints in automated workflows, ensuring critical decisions receive proper review before execution.

1

Pipeline Pauses

Execution halts at the input step, waiting for user interaction.

2

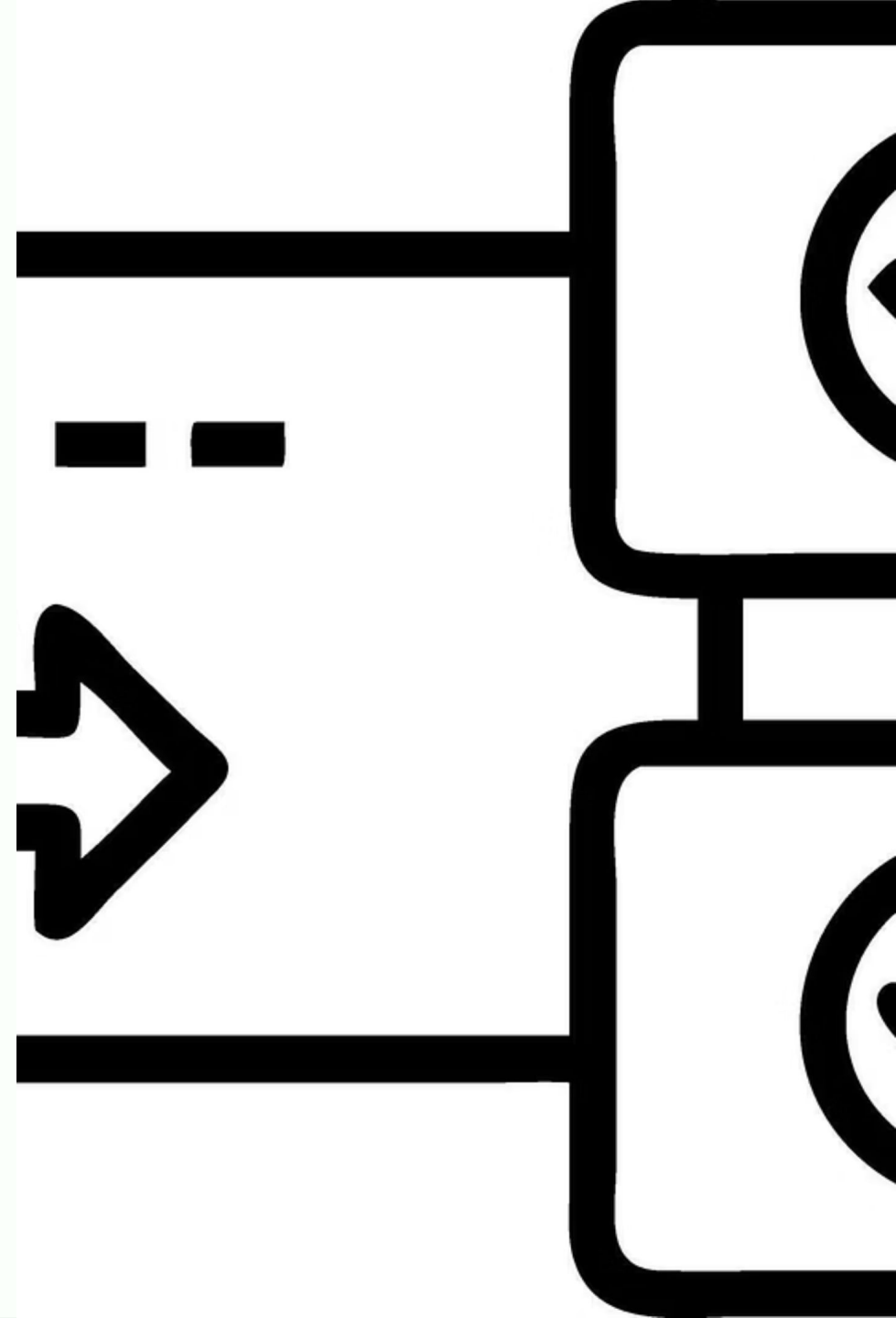
User Reviews

Team members evaluate the request and provide necessary input or approval.

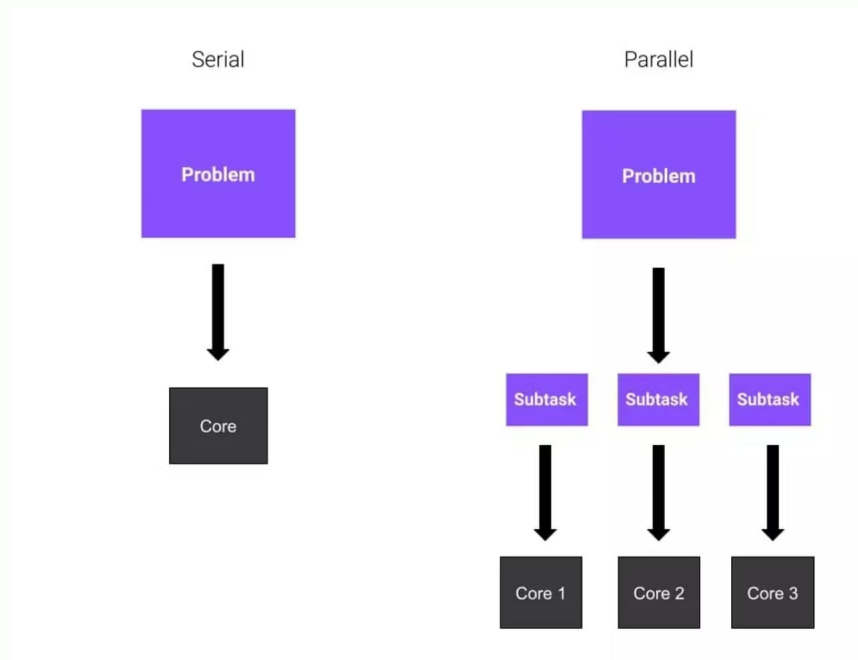
3

Pipeline Continues

Upon approval, the pipeline resumes with the next stage or step.



Parallel Execution Benefits



Maximize Efficiency

Parallel execution dramatically reduces pipeline duration by running independent tasks simultaneously.



Run tests across multiple platforms concurrently



Build different components simultaneously



Deploy to multiple environments at once

Pipeline Best Practices

Keep Pipelines Simple

Write clear, maintainable code. Break complex workflows into smaller, reusable components and shared libraries.

Use Version Control

Store Jenkinsfiles in source control alongside application code. Track changes and enable team collaboration.

Implement Error Handling

Add try-catch blocks and post-build actions to handle failures gracefully and ensure proper cleanup.

Optimize Performance

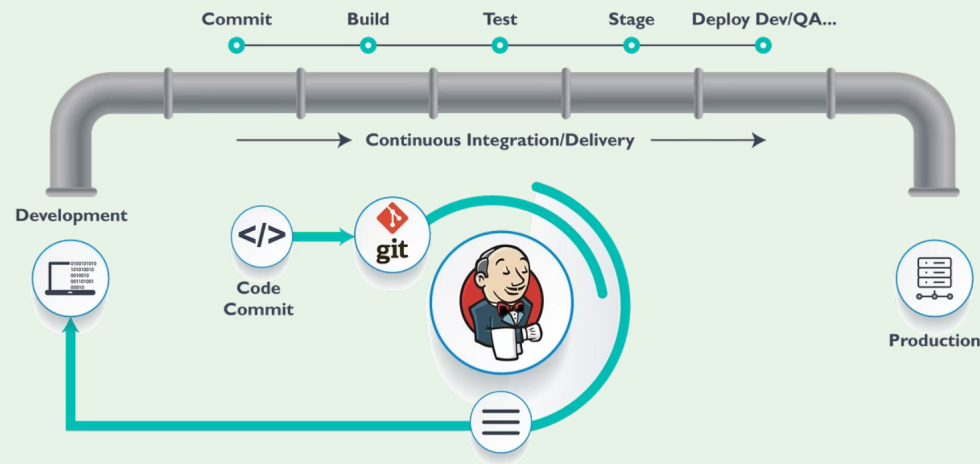
Leverage parallel execution, cache dependencies, and use appropriate agent labels to minimize build times.

Secure Credentials

Never hardcode secrets. Use Jenkins credentials management and environment variables for sensitive data.

Monitor and Iterate

Regularly review pipeline metrics, identify bottlenecks, and continuously improve your automation workflows.



Master Jenkins Pipelines

Transform your CI/CD workflows with powerful automation, efficient execution, and industry best practices.

Choose the Right Job Type

Select from Freestyle, Maven, or Pipeline projects based on your needs.

Leverage Pipeline Features

Use stages, steps, parallel execution, and input steps for sophisticated workflows.

Follow Best Practices

Keep code simple, secure credentials, and continuously optimize performance.